

ТАКТ — бэклог улучшений по принципам антихрупкости

Документ фиксирует **оставшиеся** улучшения APM SOC-оператора после внедрения топ-5. Направление — антихрупкость по Талебу: система должна не просто «переживать» стресс, а становиться от него лучше. Каждый пункт — с обоснованием, объёмом работ и критерием приёмки.

Статус ветки: `feature/takt-pt-frontend-dark-ui` . Стенд демонстрационный, состояние движка
(`stand/engine_state.py`) держится **в памяти процесса** — это первое, что нужно закрыть для любого использования дольше одной демо-сессии.

Уже внедрено (топ-5 — для контекста, не переделывать)

#	Улучшение	Принцип	Где
1	Петля обучения по вердикту (TP/FP/benign → пересчёт весов инвариантов → каскад на связанные кейсы)	Skin in the game	engine_state.record_verdict, VerdictPanel.tsx
2	Честное состояние канала (LIVE/STALE/POLL/DOWN) + heartbeat + REST-фолбэк	Via negativa, без «ложного зелёного»	client.ts subscribeToUpdates, CommandBar.tsx
3	Разделение риск / ОТ-импакт / доверие + маркер «тихого хвоста»	Barbell	synthetic_data.py, CaseWorkbench.tsx
4	Хоткеи триажа (1-4 / a / e /// x) + эксклюзивный лок (409)	Оптимизация под оператора	IncidentQueue.tsx, engine_state.lock
5	Инъекция хаоса (6 режимов, деградация без падения)	Гормезис	ChaosPanel.tsx, server.py /chaos

Оставшиеся улучшения

P0 — фундамент (без этого «обучение» теряется при рестарте)

В-1. Персистентность состояния и аудита

- **Проблема.** Веса инвариантов, вердикты, локи и hash-chain аудит живут в памяти. Рестарт пода — и всё «обучение» оператора обнуляется. Skin in the game без памяти бессмысленен.
- **Что сделать.** Вынести EngineState в SQLite (WAL) или Postgres: таблицы weights, verdicts, locks, audit (append-only). Загрузка при старте, атомарная запись при каждом вердикте. Hash-chain аудита писать на диск и проверять целостность при загрузке.
- **Файлы.** stand/engine_state.py (слой хранилища), stand/server.py (lifespan), миграции.
- **Приёмка.** FP по кейсу → рестарт сервера → веса и вердикт сохранились; /api/v1/audit проходит проверку целостности цепочки хэшей.

- **Объём.** ~1 день.

В-2. Защита обучения от отравления (guardrails + откат)

- **Проблема.** Один оператор (или скомпрометированная учётка) серией FP может «занулить» веса ключевых инвариантов и ослепить SOC. Это классическая adversarial-поверхность петли обучения.
- **Что сделать.**
 - лимит суммарного дрейфа веса за смену (напр. не более $\pm 40\%$ от baseline на инвариант);
 - аномалия на `calibration_delta` (резкий скачок → требовать подтверждение L2/4-eyes);
 - кнопка «Откатить вердикт» — снимает эффект конкретного вердикта и пересчитывает веса;
 - «карантин» инварианта: если по нему за окно $>N$ противоречивых вердиктов — заморозить вес и вынести на ревью, а не двигать дальше.
- **Файлы.** `engine_state.record_verdict` (лимиты), новый `revert_verdict`, `VerdictPanel.tsx`.
- **Приёмка.** 10 FP подряд по одному инвариантному семейству → вес упирается в нижний лимит и инвариант помечается «на ревью»; «Откатить» возвращает предыдущее состояние весов.
- **Объём.** ~1-1.5 дня. (Antifragility: система выдерживает злонамеренный ввод и учится отличать сигнал от атаки на сам процесс обучения.)

P1 — операционная зрелость

В-3. Мультиоператор и передача смены

- **Проблема.** `OPERATOR_ID` захардкожен (`operator.shift-A`). Локи и вердикты не различают людей; нет передачи смены.
- **Что сделать.** Идентификация оператора (заголовок/сессия), presence-индикатор (кто в очереди сейчас), TTL на лок с авто-снятием (**dead-man's switch** — «завис» оператор не блокирует кейс навечно), заметка при передаче смены (handoff note в аудит).
- **Файлы.** `server.py` (проброс operator), `engine_state.lock` (TTL), новый `ShiftHandoff`.
- **Приёмка.** Два разных operator-id: А взял кейс → В видит «занято А» (409); лок сам снимается через TTL; в аудите видна запись передачи смены.
- **Объём.** ~1 день.

В-4. Контрфактический реплей (бэктест обучения)

- **Проблема.** Оператор не видит, насколько его вердикты улучшили систему — обучение «на веру».
- **Что сделать.** Прогон исторического потока событий против **текущих** весов: сколько прошлых FP теперь были бы подавлены, не задело ли это истинные срабатывания (защита от переобучения). Экран «Эффект калибровки»: до/после на исторических данных.
- **Файлы.** новый `engine_state.replay()`, эндпоинт `/api/v1/replay`, экран во фронтенде.
- **Приёмка.** После серии FP реплей показывает снижение шума на N% при 0 подавленных TP из эталонной цепочки.
- **Объём.** ~1.5 дня. (Via negativa: проверяем, что «улучшение» не выкинуло сигнал.)

В-5. Барбельный авто-триаж по доверию

- **Проблема.** Low-risk/low-impact/low-confidence шум всё равно занимает внимание.
- **Что сделать.** Авто-подавление таких кейсов, **но** с обязательной случайной выборкой (напр. 5%) на ручной обзор — чтобы не ослепнуть на «тихом» классе (barbell: почти всё автоматизируем, малую долю намеренно смотрим руками). Порог — функция доверия, не риска.
- **Файлы.** `engine_state.list_cases` (флаг `auto_suppressed + sampling`), UI-фильтр «показать подавленные».
- **Приёмка.** Шум ниже порога уходит из основной очереди; ~5% остаётся видимым со стемпом «контрольная выборка»; ни один tail-risk кейс не подавляется автоматически.
- **Объём.** ~0.5 дня.

В-6. Вахта за «тихим хвостом» ОТ

- **Проблема.** Маркер tail-risk есть, но нет процесса: низковоероятный/катастрофичный по ОТ-импакту кейс может «протухнуть» внизу очереди.
- **Что сделать.** Отдельный watchlist tail-risk с обязательным периодическим ревью (если кейс не тронут X часов — принудительная эскалация/напоминание). Счётчик «возраст без обзора».
- **Файлы.** `IncidentQueue.tsx` (вкладка «Хвост ОТ»), `engine_state` (aging), фон-проверка.
- **Приёмка.** Tail-кейс без действий > порога → подсветка + запись эскалации в аудит.
- **Объём.** ~0.5 дня.

Р2 — устойчивость и доказуемость

В-7. Хаос как регламентная «game day» + scorecard

- **Проблема.** Хаос сейчас ручной. Гормезис работает, когда стресс регулярный и измеряемый.
- **Что сделать.** Планировщик хаос-прогонов (напр. авто-режим на 2 мин раз в час в демо), scorecard: MTTR под хаосом, доля переживших запросов, «сломалось/деградировало». Гейт в CI: прогнать хаос-сценарии и упасть, если сервер вернул 5xx или UI «белый экран».
- **Файлы.** `stand/benchmark.py` (scorecard), `tests/test_chaos_resilience.py`, CI-job.
- **Приёмка.** CI-джоб гоняет все 6 режимов хаоса, ассертит «нет 5xx, SSE восстановился, клиент не упал»; отчёт scorecard в артефактах сборки.
- **Объём.** ~1 день.

В-8. Подписанный экспорт аудита (комплаенс)

- **Проблема.** Hash-chain аудит есть, но нет выгрузки для внешней проверки.
- **Что сделать.** Экспорт аудита как подписанного артефакта (JSON + подпись/НМАС) и эндпоинт верификации целостности. Каждый вердикт/лок/эскалация — прослеживаемы.
- **Файлы.** `/api/v1/audit/export`, `/api/v1/audit/verify`.
- **Приёмка.** Экспорт → изменение одной записи → verify сообщает о разрыве цепочки.
- **Объём.** ~0.5 дня.

В-9. Адаптер реальных детекторов (путь в прод)

- **Проблема.** Инвариантный движок питается синтетикой.
- **Что сделать.** Слой нормализации входа (Sigma/OCSF/ECS → внутренний формат события и маппинг на инварианты `INV-*`), чтобы петлю обучения можно было включить на реальном потоке.
- **Файлы.** новый `stand/adapters/` (или отдельный модуль), контракт события.
- **Приёмка.** Тестовый Sigma-хит нормализуется и создаёт кейс с корректными `invariants`.
- **Объём.** ~2 дня.

РЗ — качество и UX

В-10. Тесты и наблюдаемость петли обучения

- Property-based тесты на клэмп весов (0.2–1.5) и монотонность (FP не повышает вес, TP не понижает); нагрузочный тест SSE fan-out (heartbeat под N клиентами); e2e Playwright на сценарии вердикт → каскад, хаос → фолбэк, лок → 409. Метрики Prometheus по петле (кол-во вердиктов, средний дрейф, MTTR под хаосом).
- **Приёмка.** `pytest` + Playwright зелёные в CI; `/metrics` отдаёт счётчики петли.
- **Объём.** ~1.5 дня.

В-11. Command palette (Ctrl-K) и доступность

- Единая палитра команд поверх хоткеев; ARIA-live для состояния канала (STALE/POLL озвучивается скринридером — честность канала должна быть доступной, а не только визуальной); контраст под дежурство в тёмном помещении.
- **Объём.** ~1 день.

Порядок внедрения (рекомендация)

1. **В-1** (персистентность) — без неё обучение эфемерно.
2. **В-2** (guardrails/откат) — сразу после, иначе персистентная петля уязвима к отравлению.
3. **В-3, В-6** (мультиоператор, вахта за хвостом) — операционная готовность к пилоту.
4. **В-7, В-10** (хаос в CI + тесты) — institutionalize гормезис и зафиксировать регрессии.
5. **В-4, В-5, В-8, В-9, В-11** — по мере приоритетов заказчика.

Быстрые победы (< 1 дня): В-5, В-6, В-8, В-2 (частично — только «Откатить вердикт»).